Battery Model As First Case Study

A compact validation case before the nonlinear cookie-factory energy system

Why include it

- Clean storage use case: electricity-price arbitrage with SOC and power limits
- Shows the same thesis question in a simpler, explainable setting

MDP formulation

- State: SOC, current price, time, feasible charge/discharge limits, 24h forecast
- Action: continuous a in [-1, 1], scaled to battery power
- Reward: -grid cost – (infeasible-action penalty)
- Dynamics: SOC(t+1) = SOC(t) + P_actual * dt / E_max

Experimental design

- Keep SAC, model architecture, training budget and price windows fixed
- Train each reward variant over multiple seeds
- Evaluate on unseen 24h windows
- Compare against Mathematical/IPOPT baseline
- Use the same metrics for battery and cookie-factory studies

Battery Model Mathematical Formulation

Objective Function

$$\min \sum_{t \in T} \text{price}_t \; P_t \quad \text{.....(1)}$$

Power Gating Constraint

$$P_t = P_{max} \; (\alpha_t \quad u_t \quad \beta_t + \gamma_t \quad u_t \quad \delta_t) \quad \text{.....(2)}$$

Where the smoothing terms are defined as:

$$\alpha_t = 0.5\,(1 + \tanh\,(k(u_t - 0.0025)))$$
$$\beta_t = 0.5\,(1 + \tanh\,(k(0.995 - \text{SOC}_t)))$$
$$\gamma_t = 0.5\,(1 - \tanh\,(k(u_t + 0.0025)))$$
$$\delta_t = 0.5\,(1 + \tanh\,(k(\text{SOC}_t - 0.005)))$$

Actual Power and SOC Dynamics

$$P_{actual,t} = \eta \quad P_t \quad \text{.....(3)}$$

$$\text{SOC}_t = \text{SOC}_{t-1} + \Delta t \quad \frac{P_{actual,t}}{E_{max}} \quad \forall t \in T, t > 0$$

$$\text{SOC}_0 = \text{SOC}_{initial} + \Delta t \quad \frac{P_{actual,0}}{E_{max}}$$

Decision Variables & Bounds

Four variables are defined at each time step $t = 0, \ldots, T-1$, each with explicit physical bounds:

1

Control Input u_t

Normalized charge/discharge command.

$$u_t \in [-1, 1]$$

2

State of Charge SOC_t

Fractional energy level of the battery.

$$\mathrm{SOC}_t \in [0, 1]$$

3

Grid Power P_t

Scheduled power exchange with the grid.

$$P_t \in [-P_{\max}, P_{\max}]$$

4

Effective Power P_t^{act}

Battery power after efficiency losses.

$$P_t^{\mathrm{act}} \in [-\eta P_{\max}, \eta P_{\max}]$$

Sign convention: $P_t > 0$ → grid import (cost); $P_t < 0$ → grid export (revenue).

Objective: Minimize Electricity Cost (1)

Minimize total energy procurement cost over the horizon T:

$$\min_{u,\,\mathrm{SOC},\,P,\,P^{\mathrm{act}}} \sum_{t=0}^{T-1} \pi_t \, P_t$$

where $\boldsymbol{\pi_t}$ is the electricity spot price at step t. The optimizer exploits low-price periods to charge and high-price periods to discharge, reducing net grid cost.

Variable Legend

- $\boldsymbol{\pi_t}$ — electricity price at time t
- $\boldsymbol{P_t > 0}$ — import; pay $\pi_t.Pt$
- $\boldsymbol{P_t < 0}$ — export; earn $|\pi_t.Pt|$
- $\boldsymbol{T}$ — optimization horizon length

Constraints: Smooth power mapping (2)

Non-differentiable switching at SOC limits is approximated by smooth tanh-based gates, enabling gradient-based solvers.

$$\sigma_k(x) = \frac{1}{2}(1 + \tanh(kx))$$

The gated power map becomes:

$$P_t = P_{\max}(\underbrace{\sigma_k(u_t - \varepsilon)\,\sigma_k(0.995 - \mathrm{SOC}_t)}_{\text{charging branch}}\, u_t + \underbrace{\sigma_k(-u_t - \varepsilon)\,\sigma_k(\mathrm{SOC}_t - 0.005)}_{\text{discharging branch}}\, u_t)$$

$\sigma_k(u_t - \varepsilon)$

Activates charging branch when $u_t > 0$

$\sigma_k(-u_t - \varepsilon)$

Activates discharging branch when $u_t < 0$

$\sigma_k(0.995 - SOC_t)$

Suppresses charging near full ($SOC \approx 1$)

$\sigma_k(SOC_t - 0.005)$

Suppresses discharging near empty ($SOC \approx 0$)

Parameters: dead-band offset $\varepsilon = 0.0025$; steepness k controls sharpness of the gate transition. Larger k approaches a hard switch.

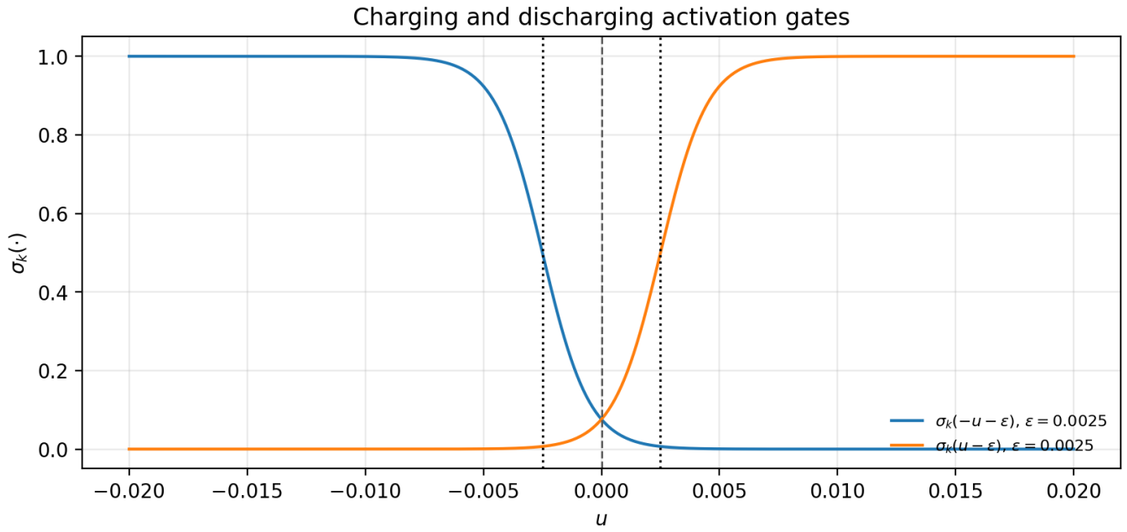


Fig. 1: Charging and discharging activation gates with a small deadband around u = 0.

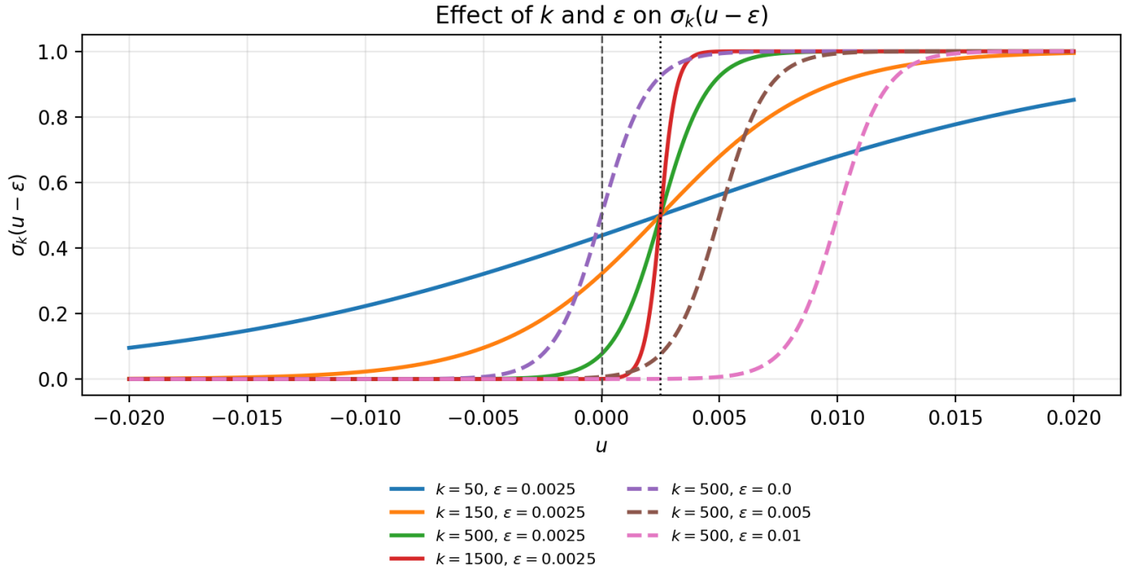


Fig. 2: Effect of k and epsilon on gate sharpness and threshold shift.

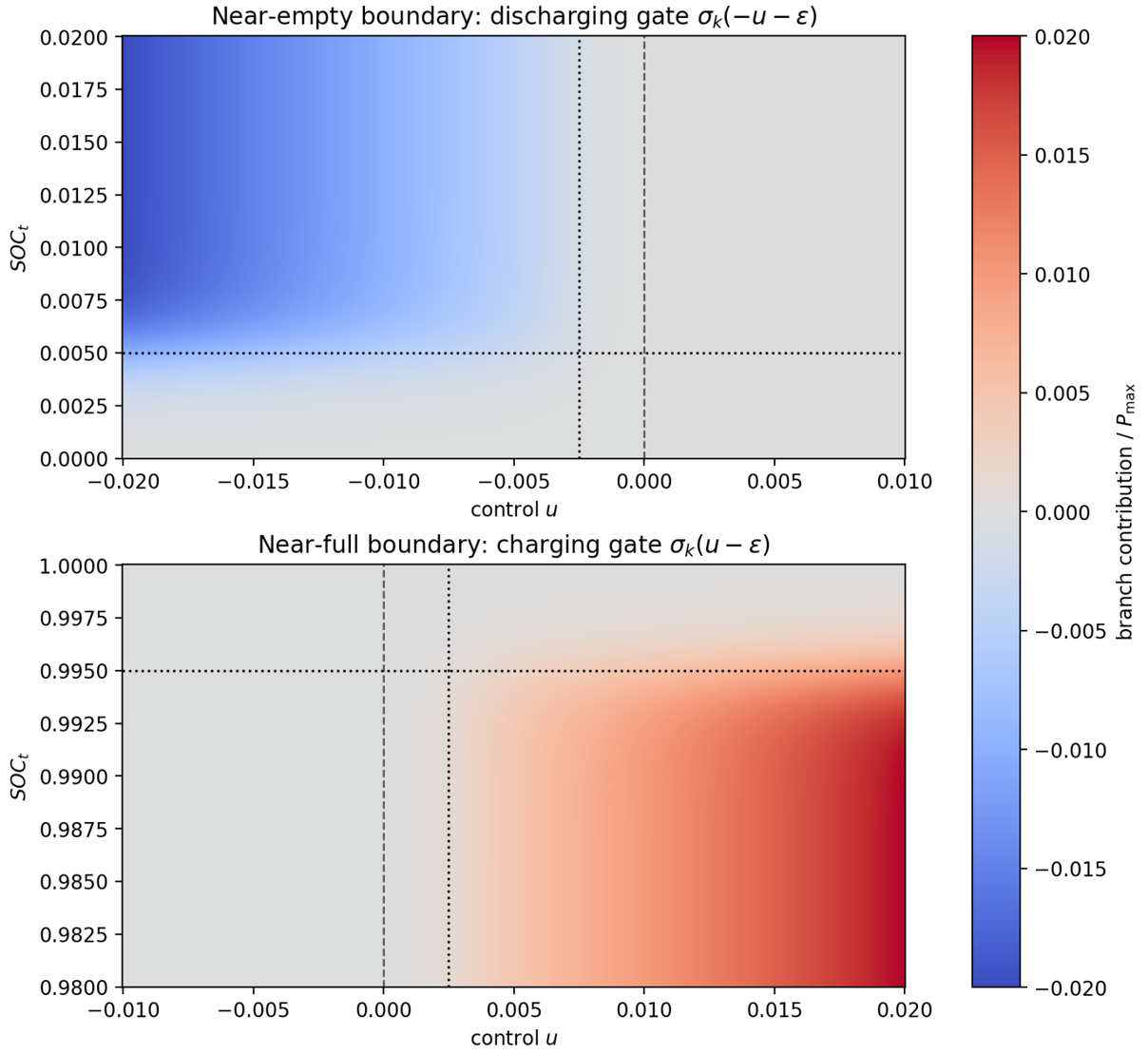


Fig. 3: SOC boundary behavior, suppressing discharge near empty and charge near full.

Constraints: Efficiency Model & SOC Dynamics (3)

Efficiency Map

A factor η is applied to both charge and discharge:

$$P_t^{\text{act}} = \eta\, P_t$$

Implementation uses η = *0.9.*

State-of-Charge Recursion

With Δt = 3600 *s* and capacity E_{max}:

$$\text{SOC}_0 = \text{SOC}_{\text{init}} + \frac{\Delta t}{E_{\max}} P_0^{\text{act}}$$

$$\text{SOC}_t = \text{SOC}_{t-1} + \frac{\Delta t}{E_{\max}} P_t^{\text{act}}, \quad t = 1, \ldots, T-1$$

Each step integrates effective battery power into the energy state, enforcing causality across the horizon.

Why Soft Actor-Critic (SAC)?

The control task needs continuous setpoints, sample-efficient learning, and stable exploration under changing electricity prices

Algorithm	Why it was not the main choice
Q-learning	Works best for small discrete states/actions; continuous battery power would need coarse discretization.
DQN	Deep version of Q-learning, but still mainly discrete-action;
PPO	Can handle continuous actions and is stable, but on-policy learning usually needs more fresh simulation data.
TD3	Strong deterministic baseline, but SAC gives more deliberate exploration through entropy regularization.

Why SAC fits this thesis

- Continuous actor outputs the charge/discharge setpoint directly
- Off-policy replay reuses simulation data efficiently
- Entropy term keeps exploration alive during price spikes and SOC trade-offs
- After training, inference is a direct state-to-action mapping
- Same algorithm family can scale from battery power to cookie-factory continuous controls

Analysis of RL Battery Optimization Box Plot (32 Runs)

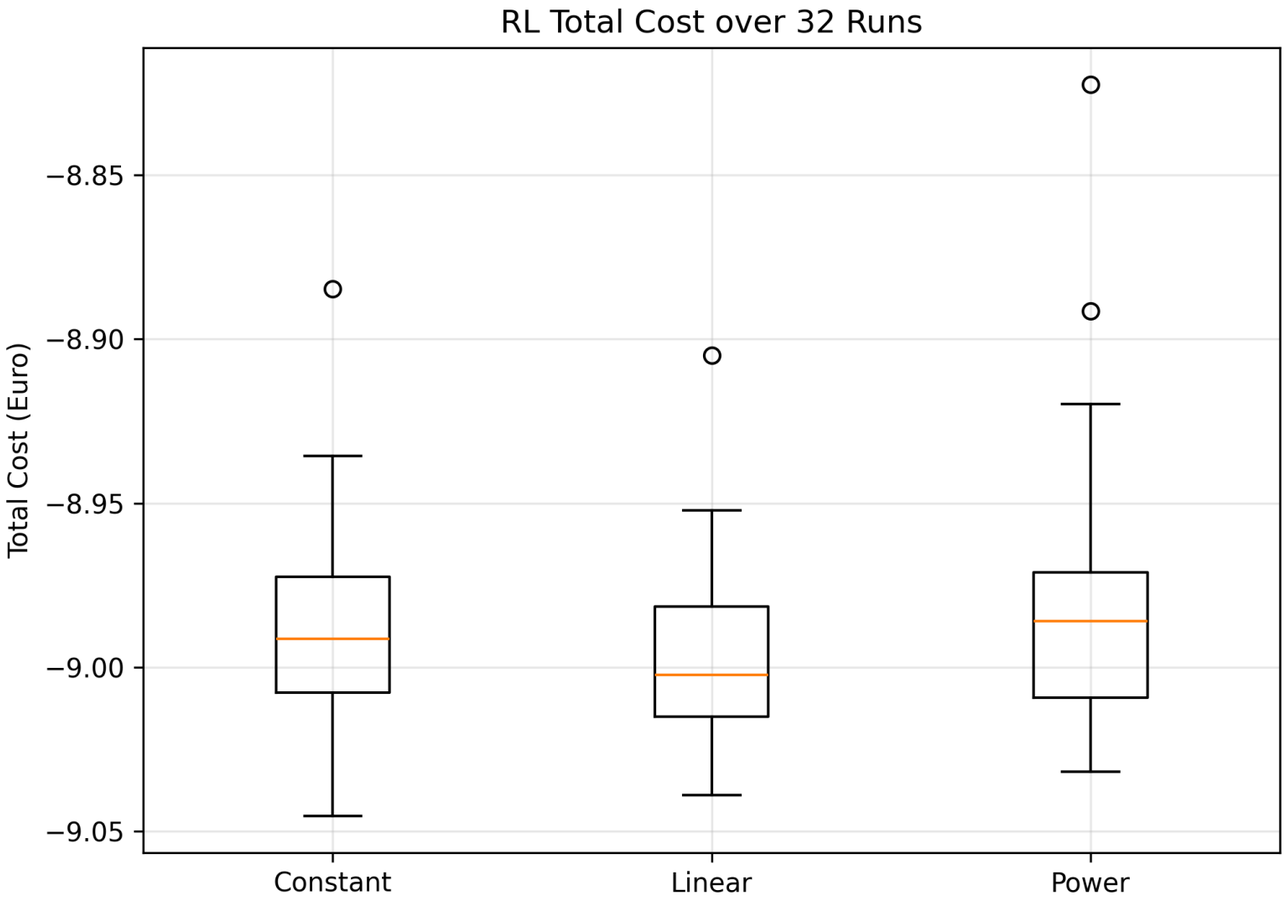


Fig.: Distribution of total cost from 32 RL models trained with identical hyperparameters and evaluated over the same time horizon.

* Mathematical Optimization Cost: -10.12 €

Analysis of RL Battery Optimization Box Plot (32 Runs)

The linear learning rate schedule performs best because it achieves the lowest median cost and maintains a tight distribution across runs, indicating stable and reliable convergence. The linear decay allows large updates during early exploration and smaller updates for fine policy refinement later in training.

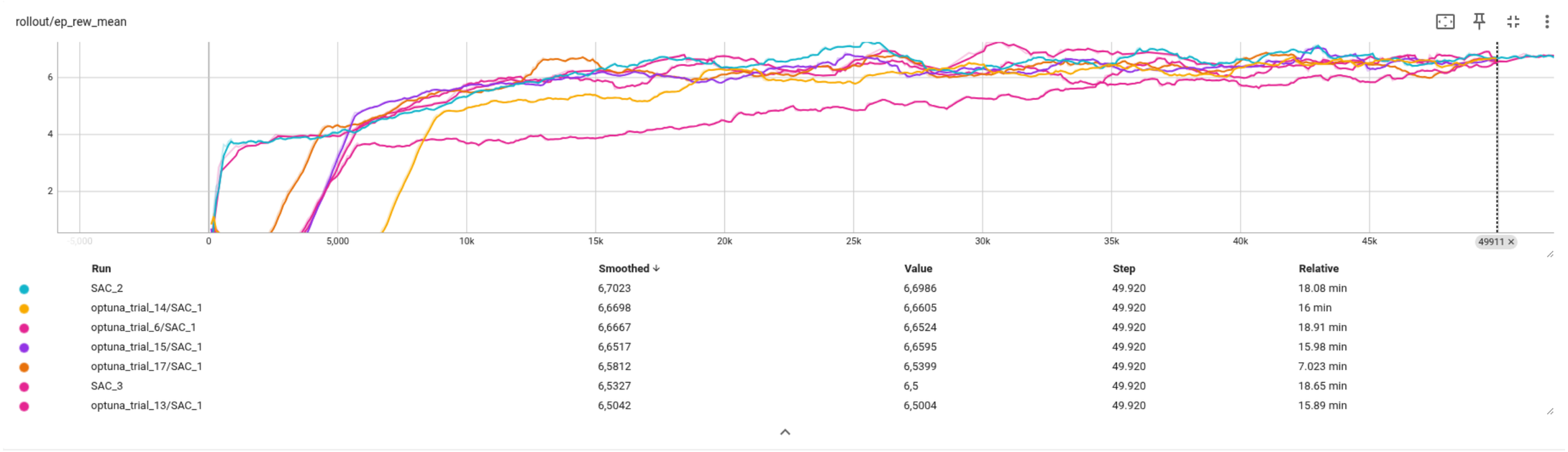
Strengths:

- 31/32 runs (96%) achieved near-optimal performance
- Stable and consistent learning behavior
- Model is robust to random initialization

Weaknesses:

- 4% chance of suboptimal convergence (1 outliers)
- Optimality of the learned policy cannot be guaranteed

Hyperparameter Tuning: Training Performance Comparison



Run	Smoothed	Value	Step	Relative
SAC_2	6,7023	6,6986	49.920	18.08 min
optuna_trial_14/SAC_1	6,6698	6,6605	49.920	16 min
optuna_trial_6/SAC_1	6,6667	6,6524	49.920	18.91 min
optuna_trial_15/SAC_1	6,6517	6,6595	49.920	15.98 min
optuna_trial_17/SAC_1	6,5812	6,5399	49.920	7.023 min
SAC_3	6,5327	6,5	49.920	18.65 min
optuna_trial_13/SAC_1	6,5042	6,5004	49.920	15.89 min

Fig: Comparison of episode rewards across multiple SAC training runs with different hyperparameter configurations.

Computational Time Comparison

Method	Time
RL Training	≈ 16 minutes
RL Policy Execution	0.02 seconds
Optimization	0.16 seconds

Although RL requires higher upfront training time, the decision-making during operation is significantly faster than classical optimization, making RL suitable for real-time control applications.

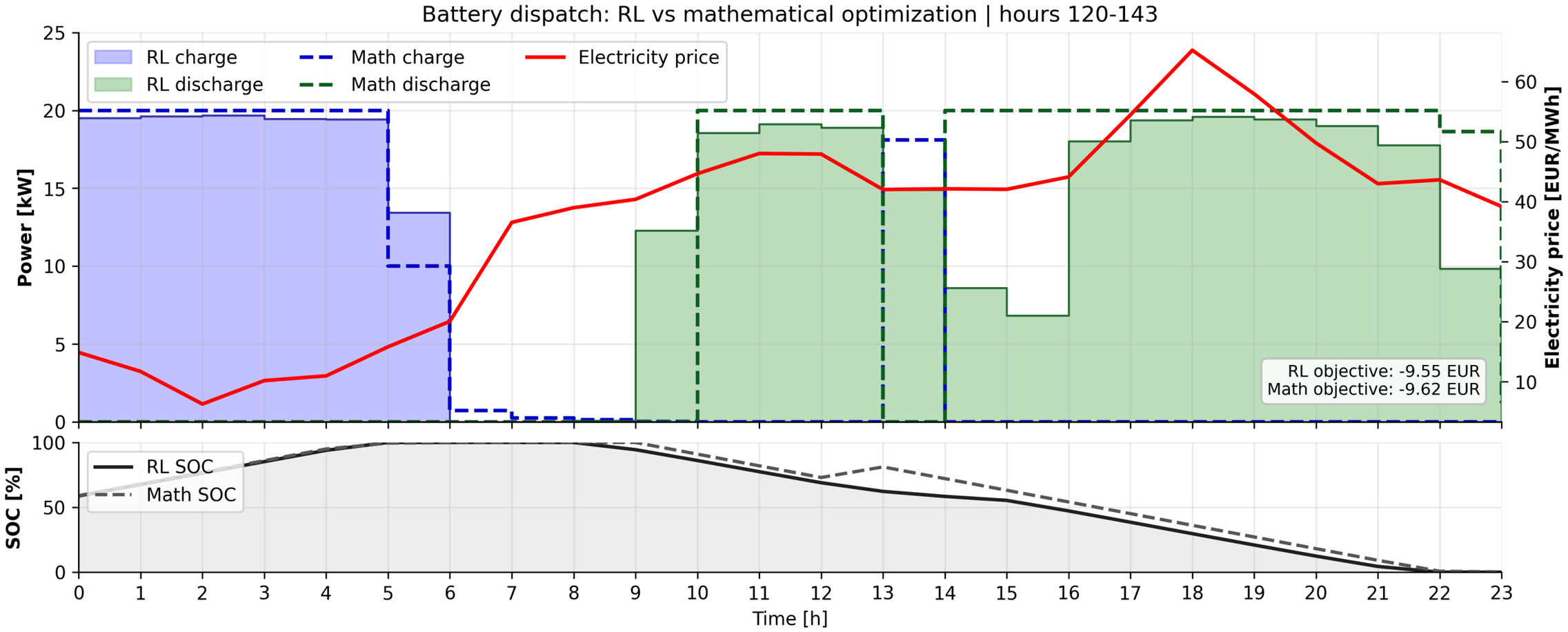
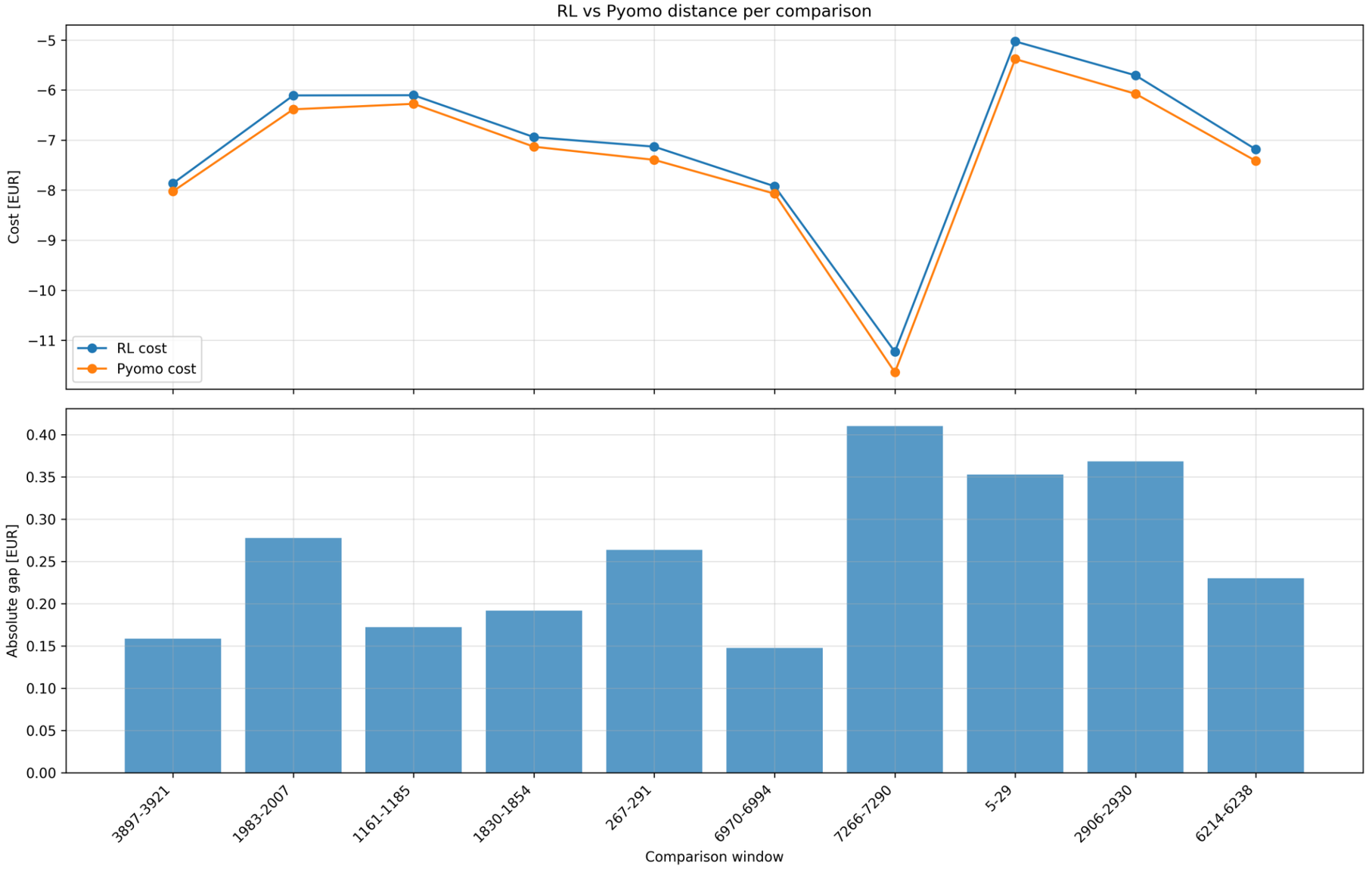


Fig: Visualization of system dynamics and control for one trajectory.

Multiple Runs: Inference Performance Comparison



Thank You!

I appreciate your time and attention.